

Arduino 4 — Non-Blocking Timing with millis()

Tutorial Group: Arduino

Goal

In *Arduino 1 — Install the Arduino IDE and Run the Blink Test*, you used `delay()` to control how quickly the LED blinked. In this tutorial, you will replace `delay()` with the Arduino `millis()` function. The technique allows the Arduino to keep executing `loop()` while keeping track of how much time has passed.

You will first use this technique to blink one LED, and then extend the program to control two LEDs at different rates.

What You Need

- Arduino Mega 2560
- USB cable
- Computer with the Arduino IDE
- Breadboard and jumper wires
- One external LED and 220 Ω resistor from the Mega 2560 evaluation kit

Part 1 — Understanding millis()

The Arduino provides a function called:

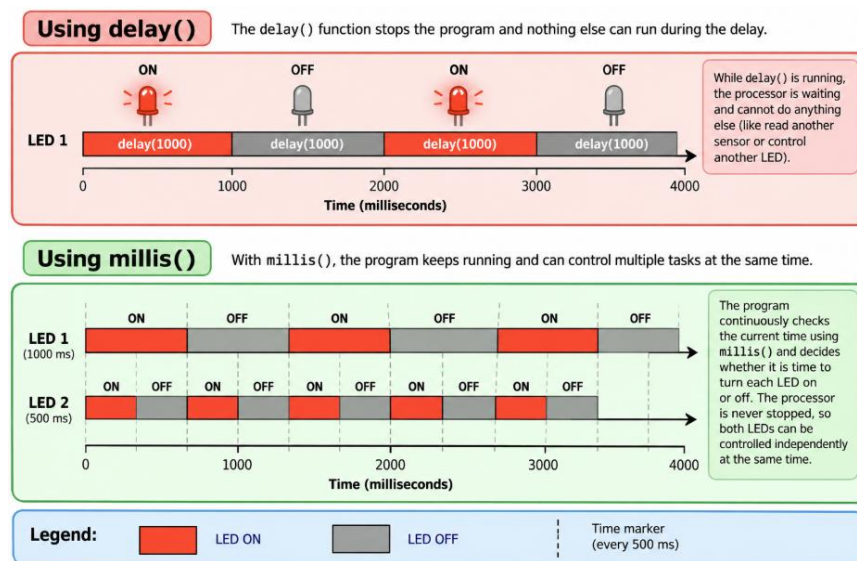
```
millis()
```

`millis()` returns the number of milliseconds that have elapsed since the Arduino was started. For example, approximately 1 second after startup it will return 1000, and approximately 5 seconds after startup it will return 5000.

Unlike `delay()`, calling `millis()` does not stop the program. For example:

```
unsigned long currentTime = millis();
```

simply asks the Arduino how many milliseconds have elapsed. The program immediately continues executing as shown in this diagram:



Part 2 — Remember When the LED Was Last Updated

The program needs to remember when the LED was last changed. Create a global variable near the beginning of your program:

```
unsigned long lastLedUpdate = 0;
```

This variable contains the value of `millis()` from the last time the LED changed state. It is initialized to 0 because when the Arduino starts, the LED has not yet been updated by this program.

The variable must be global because its value needs to survive from one execution of `loop()` to the next. If it were declared inside `loop()`, it would be recreated every time `loop()` executes.

Part 3 — Decide How Often to Update the LED

Create a global constant containing the desired time between LED changes:

```
const unsigned long ledInterval = 1000;
```

The value is in milliseconds, so 1000 milliseconds is 1 second. The `const` keyword tells the compiler that this value should not be changed by the program.

Using a named constant makes the program easier to understand and modify. Changing the value to 500 would make the LED change state every 500 milliseconds.

Part 4 — Keep Track of the LED State

Create another global variable:

```
bool ledState = LOW;
```

The program can flip the state using:

```
ledState = !ledState;
```

The `!` operator means NOT. Therefore, LOW becomes HIGH and HIGH becomes LOW.

Part 5 — Get the Current Time

Inside `loop()`, get the current value of `millis()`:

```
unsigned long currentTime = millis();
```

Every time `loop()` executes, `currentTime` contains the current elapsed time. The loop may execute many thousands of times while waiting for the next LED transition.

Part 6 — Determine Whether It Is Time to Change the LED

The elapsed time since the last LED update is calculated by subtracting the previous time from the current time:

```
currentTime - lastLedUpdate
```

If that value is greater than or equal to the desired interval, it is time to change the LED:

```
if ((currentTime - lastLedUpdate) >= ledInterval)
{
    // Update the LED
}
```

Think of the calculation as: current time – time of last update = time elapsed since last update.

Part 7 — Update the LED and Record the Time

Inside the if statement, three things need to happen.

- Flip the LED state.
- Send the new state to the LED.
- Remember when the change was made.

```
ledState = !ledState;
digitalWrite(LED_BUILTIN, ledState);

lastLedUpdate = currentTime;
```

Updating lastLedUpdate is important. Without it, the condition would remain true after the first interval and the LED would change state on every pass through loop().

Part 8 — Complete One-LED Program

Put the pieces together:

```
const int ledPin = LED_BUILTIN;

unsigned long lastLedUpdate = 0;
const unsigned long ledInterval = 1000;

bool ledState = LOW;

void setup()
{
    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, ledState);
}

void loop()
{
    unsigned long currentTime = millis();

    if ((currentTime - lastLedUpdate) >= ledInterval)
    {
        ledState = !ledState;
        digitalWrite(ledPin, ledState);

        lastLedUpdate = currentTime;
    }
}
```

Upload the program and verify that the LED changes state approximately once per second. There is no delay() anywhere in the program.

Part 9 — What Is the Arduino Doing While It Waits?

This is the most important concept in this tutorial. After the LED changes, `loop()` immediately starts executing again. Until the interval has elapsed, the `if` statement does nothing and `loop()` continues.

The Arduino never stopped to wait for the second to pass. It repeatedly checks the time and continues doing other work.

Part 10 — Two Independently Timed LEDs

Now extend the program to control a second LED. Suppose LED 1 should change state every 1000 ms and LED 2 every 500 ms.

With `delay()`, the sketch is blocked while it waits and is unable to do any additional processing. With `millis()`, each LED can have its own timer. This is the key advantage of using `millis()` for delays: the program can check both timers every time `loop()` executes.

Extend the one-LED program to control a second LED. Use a 1000 ms interval for LED 1 and a 500 ms interval for LED 2 and connect the second LED to digital pin 8 through a 220 Ω resistor, with the LED cathode connected to GND.

You will need two independent sets of timing information:

- last-update variable
- interval constant
- LED state variable
- digital output pin

Complete the second timing check yourself. Do not use `delay()`.

Part 11 — Give Each LED Its Own Timer

The first LED needs:

```
unsigned long lastLed1Update = 0;
const unsigned long led1Interval = 1000;
```

Create a second pair for LED 2:

```
unsigned long lastLed2Update = 0;
const unsigned long led2Interval = 500;
```

Each LED also needs its own state variable:

```
bool led1State = LOW;
bool led2State = LOW;
```

Part 12 — Check Both Timers in `loop()`

Both timers must be checked every time `loop()` executes. Use two separate `if` statements, not an `if` followed by `else`:

```
void loop()
{
    unsigned long currentTime = millis();

    // Check LED 1 timer
    if ((currentTime - lastLed1Update) >= led1Interval)
    {
```

```
    // Update LED 1
  }

  // Check LED 2 timer
  if ((currentTime - lastLed2Update) >= led2Interval)
  {
    // Update LED 2
  }
}
```

Part 13 — Why Can We Do This?

The reason we can independently control the two LEDs is not that the Arduino is running two programs at the same time. The Arduino has one processor executing one instruction at a time.

Instead, `loop()` executes very rapidly and repeatedly checks what needs to be done. On one pass, neither LED may need updating. On another pass, LED 2 may need updating. On another pass, LED 1 may need updating. Sometimes both timers may have expired and both LEDs can be updated during the same pass.

Because checking a timer takes very little time, the processor can repeatedly check both timers and give the appearance that the LEDs are operating independently.

This same technique can be extended to many activities: checking buttons, reading sensors, updating displays, and managing additional timed events. None of these activities needs to stop the processor while waiting for something to happen.

Part 14 — Experiment

Once your two LEDs are working, experiment with different intervals:

- LED 1 = 1000 ms, LED 2 = 200 ms
- LED 1 = 750 ms, LED 2 = 1250 ms

Observe how the LEDs continue to operate independently. Remember that the interval is the time between state changes, so a 1000 ms value produces a 1 second ON and a 1 second OFF interval.

Add a third LED. Give it its own digital output pin, state variable, last-update variable, and timing interval. Choose an interval different from the first two LEDs. The three LEDs should operate independently, with no `delay()` statements anywhere in the program.

What You Should Understand

- What `millis()` returns.
- Why the last-update variables must retain their values between executions of `loop()`.
- Why the timing intervals are declared as `const`.
- How elapsed time is calculated.
- How an if statement determines when an event should occur.
- How to change an LED state on a regular interval without using `delay()`.
- Why independently timed activities require independent timing variables.
- Why `millis()` allows multiple independently timed activities to be handled by the same `loop()`.

The key concept is: Don't wait for time to pass. Keep executing `loop()` and check whether enough time has passed for each task.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I understand what `millis()` returns.

- ☐ I understand why lastLedUpdate must be a global variable.
- ☐ I understand why the update interval is a const.
- ☐ I can explain what $((currentTime - lastLedUpdate) \geq ledInterval)$ does.
- ☐ I can blink one LED without using delay().
- ☐ I can explain why the LED state must be stored.
- ☐ I can independently control two LEDs at different rates.
- ☐ I understand why each LED needs its own timing variables.
- ☐ I understand why millis() allows multiple independently timed tasks to be handled by one loop().

If you can complete these tasks and explain how the program independently times the LEDs, you are ready for the next tutorial.